

Laborator 3

Dezvoltarea și depanarea programelor C sub Linux

3.1 Scopul lucrării

În acest laborator se va discuta dezvoltarea programelor C pe Linux, precum și rularea și depanarea acestora.

3.2 Fundamente teoretice

3.2.1 Compilarea și rularea programelor

Limbajul C este un limbaj compilat, ceea ce înseamnă că pentru a rula un program C, acesta trebuie întâi compilat. În urma rulării se obține un program executabil.

Cel mai folosit compilator de C pe Linux este `gcc` (*Gnu Compiler Collection*), care poate fi apelat precum în exemplul de mai jos, pentru a compila fișierul `program.c`.

Compilarea unui program C

```
gcc -Wall program.c -o program
```

Pe lângă numele fișierului `.c` dat ca argument, se mai folosesc opțiunile :

- `-Wall` (*warnings=all*) Această opțiune activează afișarea tuturor avertismentelor (*eng. warning*) compilatorului. Deși un program se poate compila și cu avertismente, orice programator “care își merită pâinea” le va rezolva sau va configura compilatorul să le ignore explicit.
- `-o program` (*output*) Această opțiune ne permite să alegem numele programului executabil care va rezulta în urma compilării. Prin convenție, programele executabile pe Linux nu au extensie.

Observație: comanda minimală pentru a compila fișierul `program.c` este `gcc program.c`. În acest caz, programul compilat se va numi `a.out`.

Ca să rulăm programul pe care tocmai l-am compilat, din folderul curent, vom rula:

Rularea unui program din folderul curent

```
./program
```

Motivul pentru care adăugăm `./` înainte numelui programului e pentru a indica interpretorului de comenzi faptul că acesta se găsește în folderul curent. În mod normal, interpretorul de comenzi caută programele în folderele din variabila de mediu `$PATH`.

În cazul în care programul se blochează (cel mai probabil dintr-o eroare de programare), se poate opri folosind combinația de taste `Ctrl+C`.

3.2.2 Depanarea programelor

De multe ori, un program nu funcționează conform așteptărilor, din cauza unor erori de programare. Dacă prin inspecția codului nu se identifică sursa problemei, procesul de depanare (*eng. debugging*) se poate dovedi util.

3.2.2.1 Compilarea pentru depanare

Ca să beneficiem de cât mai multă informație în momentul în care depanăm un program, acesta trebuie compilat folosind opțiunea `-g`. Prin adăugarea acestei opțiuni, în program se adaugă informații legate de codul sursă, putând de exemplu să inspectăm valoarea unei variabile în timpul depanării.

Pentru depanarea propriu-zisă, se rulează depanatorul `gdb`, primind ca argument numele programului. În consola interactivă care apare se pot rula comenzi specifice depanatorului, ce vor fi prezentate în secțiunea următoare.

Depanarea unui program cu `gdb`

```
gdb program
```

3.2.2.2 Comenzi date la depanare

Depanatorul `gdb` încarcă programul și îl suspendă înainte de execuție, dându-ne ocazia să rulăm comenzi de depanare. Printre comenzile uzuale, suportate de `gdb`, avem:

- `break source.c:lineNo` (setarea unui *breakpoint*) - se va suspenda execuția programului atunci când se ajunge la linia `lineNo` din fișierul sursă `source.c`;
- `run` - pornește execuția programului; eventualele argumente ale programului se transmit aici;
- `continue` - reia execuția programului, care continuă până la întâlnirea unui *breakpoint*;
- `step` - execută următoarea linie de cod;
- `next` - execută următoarea linie de cod, la fel ca `step`, dar dacă aceasta este un apel de funcție, se va face tot apelul într-un singur “pas”;
- `backtrace` - afișează stiva de apeluri în punctul curent al programului;
- `print var` - afișează valoarea variabilei `var`.

3.2.2.3 Depanarea post-mortem

Pentru a investiga un program care “crapă” (*eng. crash*) în timpul execuției, e util să examinăm starea programului din momentul respectiv.

La fel ca la depanarea normală, programul se compilează în prealabil cu opțiunea `-g`, pentru adăugarea de informații referitoare la codul sursă.

Pentru a activa salvarea stării unui program atunci când crapă, se va rula comanda:

Activarea core dump

```
ulimit -c unlimited
```

Comanda de mai sus are efect doar pentru terminalul curent, în care a fost rulată (se rulează o singură dată).

Se rulează apoi programul, reproducând condițiile care l-au făcut să crape. După crash, se va observa apariția unui fișier numit `core` în folderul curent, care se poate încărca în `gdb`, rulând comanda:

Încărcarea unui core dump în `gdb`

```
gdb program core
```

În comanda de mai sus, **program** este numele programului care a crăpat.

La încărcarea unui astfel de *core dump*, execuția programului este “înghețată”. Putem examina stiva de apeluri sau examina valorile variabilelor, folosind comenzile **backtrace** sau **print**, dar nu putem avansa execuția folosind **continue**, **step** sau **next**.

În multe situații, la depanarea post-mortem putem observa o problemă tipică cum ar fi, de exemplu, dereferențierea unui pointer NULL sau o variabilă neinițializată (are o valoare care pare aleatoare). Rețineți totuși că linia care conține codul greșit poate să nu fie aceeași cu linia la care a crăpat programul.

3.2.2.4 Depanarea prin mesaje

Unii programatori preferă să afișeze mesaje în diverse puncte ale programului pentru a trasa execuția acestuia și a depista problemele. Deși utilizarea unui *debugger* este preferabilă, dacă totuși folosiți această metodă ar trebui să țineți cont de următoarele aspecte:

- Mesajele afișate pentru depanare ar trebui scrise într-un fișier separat, sau în **stderr**, pentru a le separa de conținutul afișat în mod normal de program.
- Apelul unor funcții de afișare cum ar fi **printf()** sau **fprintf()** scrie textul de afișat într-un buffer, dar e posibil ca acesta să ajungă pe ecran sau în fișier cu întârziere. Putem forța afișarea apelând funcția **fflush()**.
- Unele probleme se reproduc doar pe calculatorul unor clienți, nu și pe calculatorul pe care se testează. În astfel de cazuri, afișarea de informații într-un fișier jurnal (*eng. log*) poate fi singura variantă de a investiga problema.

3.2.3 Depistarea problemelor de tip *memory leak*

O problemă comună la dezvoltarea de programe într-un limbaj care nu beneficiază de un modul de colectare și eliberare automatizată a memoriei ce nu mai este folosită (*garbage collector*) este omiterea dealocării memoriei alocate. Utilitarul **valgrind** ne permite să depistăm astfel de probleme. Cel mai simplu mod de utilizare este de a rula **valgrind** având ca prim argument numele programului urmat, eventual, de argumentele acestuia.

Testarea unui program cu **valgrind**

```
valgrind program arg1 arg2
```

Testând precum mai sus, utilitarul **valgrind** ne anunță dacă s-au găsit probleme de tip *memory leak*. Dacă dorim să se afișeze liniile la care s-a alocat memorie care nu a mai fost eliberată, se va rula:

Testarea unui program cu **valgrind**

```
valgrind --leak-check=full --show-leak-kinds=all program arg1 arg2
```

Testarea unui program cu **valgrind** poate fi mai lentă (sau mult mai lentă, în special cu opțiunile de afișare a liniilor cu probleme) decât rularea normală a unui program.

Rețineți că putem avea probleme de tipul *memory leak* alocând direct memorie (cu **malloc()**) sau apelând funcții care alocă memorie intern (de exemplu la deschiderea unui fișier sau director).

3.3 Mersul lucrării

3.3.1 Întrebări recapitulative

1. Cu ce comandă se compilează un program C sub Linux?
2. De ce e nevoie să scriem **./** înaintea numelui unui program pentru a-l rula?
3. Ce este un breakpoint în contextul depanării?
4. Ce opțiune trebuie să adăugăm la comanda de compilare pentru ca executabilul să conțină și informații de debug?
5. Ce face utilitarul *valgrind*?

3.3.2 Probleme propuse

Primele patru probleme propuse presupun corectarea unui cod existent în timp ce ultimele două presupun scrierea de programe noi. Programele care trebuie corectate se găsesc la adresa <https://github.com/cyprioprisa/os-lab-examples/tree/master/103>.

1. (20p) În *l03p1.c* se încearcă generarea tuturor submulțimilor mulțimii $\{0, 1, \dots, N-1\}$, unde N este primit ca argument în linia de comandă, folosind operații pe biți. Reparați erorile de compilare astfel încât programul să poată fi compilat și să funcționeze corect.
2. (20p) În *l03p2.c* se citesc de la tastatură două mulțimi S_1 și S_2 ca vectori ordonați și se calculează raportul dintre cardinalul intersecției și cardinalul reuniunii, $\frac{|S_1 \cap S_2|}{|S_1 \cup S_2|}$ (numit și similaritate Jaccard). Corectați codul astfel încât rezultatul să fie corect.
3. (20p) În *l03p3.c* se citește de la tastatură un șir și se calculează suma numerelor prime din acesta. Programul crapă în timpul rulării. Utilizând tehnica post-mortem debugging prezentată mai sus, identificați și reparați problemele.
4. (25p) Programul din *l03p4.c* gestionează în memorie un vector de studenți, pe care se pot face diverse operații prin comenzi de la tastatură (*add*, *del*, *list*, *exit*). Deși funcționează corect, programul are memory leak-uri. Folosiți programul **valgrind** pentru a le identifica și reparați-le.
5. (30p) Implementați și testați funcția

```
int insert(int *v1, int n1, int c1, int *v2, int n2, int pos);
```

care inserează vectorul **v2** de lungime **n2** în vectorul **v1** de lungime **n1** și capacitate **c1**, pe poziția **pos**. Funcția va returna 0 în caz de succes și -1 în cazul în care capacitatea **c1** nu este suficient de mare.
Exemplu: dacă **v1**, de capacitate cel puțin 8 conține elementele 1, 2, 3, 4, 5, 6, iar **v2** conține elementele 10, 20, inserarea făcându-se pe poziția 3, **v1** va deveni 1, 2, 3, 10, 20, 4, 5, 6.
6. (30p) Implementați un program care primește un întreg fără semn (**unsigned int**) ca argument în linia de comandă, îl parsează (argumentele în linia de comandă sunt string-uri, iar noi avem nevoie de numere) și inversează primul byte cu ultimul și al 2-lea cu penultimul.